

FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)



COMP451 Section B

Session: Spring 2026

Assignment 2 Report

Inderias Samson - 271052262

Abstract

The objective of this project was to design and implement a custom MIPS assembler in the C programming language, capable of translating a specific subset of twelve assembly instructions into 32-bit hexadecimal machine code. While the translation of basic arithmetic is straightforward, the primary technical challenge I addressed was the "Forward Reference Problem." This occurs when an instruction, such as a jump or a branch, refers to a symbolic label (like start: or end:) that appears later in the source code. To solve this, my research question focused on **how a two-pass parsing algorithm could be structured to accurately resolve these symbolic addresses while maintaining the strict memory constraints of the MIPS architecture.**

The project's design utilizes a two-pass architecture to ensure reliability. In the first pass, my program scans the input file line-by-line to identify all labels. Mapping them to their respective memory addresses, starting at 0x00400000, and storing them in a structured Symbol Table. This pass also filters out empty lines and white spaces to prepare for translation. In the second pass, the assembler performs the actual encoding. I implemented specialized functions like `get_register` to map mnemonics (like \$t0) to their hardware indices and utilized bitwise shift and mask operations to pack opcodes, registers, and immediates into the final 32-bit format. I specifically handled the complex syntax of memory instructions like `sw` and `lw` by using `sscanf` to parse offsets and base registers separately.

The major findings of this project demonstrate that a two-pass approach is the most deterministic solution for label resolution in low-level systems. My implementation successfully handles R-Type, I-Type, and J-Type instructions, producing output that is 100% consistent with the industry-standard MARS IDE. By isolating address discovery from instruction encoding, I was able to accurately calculate relative branch offsets and absolute jump targets. The results confirm that this systematic approach effectively reduces the complexity of dynamic memory mapping, ensuring that the generated machine code is both syntactically and semantically correct for execution.

Introduction

At its core, computer programming is a series of translations. While we often write code in high-level languages that feel like English, the processor only understands a specific stream of high and low voltages represented as zeros and ones. This project focuses on a critical piece of that translation puzzle: the MIPS Assembler. An assembler acts as the final bridge between human-readable assembly mnemonics, like `add`, `sw`, or `beq`, and the hexadecimal machine code that a MIPS processor can actually execute. For this assignment, I set out to build a program in C that could take a raw text file of assembly instructions and transform it into a perfectly formatted table of memory addresses and 32-bit machine code.

The real challenge in writing an assembler isn't just swapping a word for a number; it's about understanding the spatial logic of the program. When a programmer writes a label like `start:` or `loop:`, they are creating a symbolic bookmark in memory. The computer, however, doesn't know what "start" means; it only knows that it needs to jump to a specific 32-bit address. This led to the central research question of my project: **"How can a two-pass parsing algorithm be implemented in C to reliably resolve these symbolic labels into physical memory addresses."**

Through this experiment, I expected to gain a much deeper understanding of how memory is structured. Specifically, I wanted to learn how to manipulate data at the bit level. Shifting opcodes and register indices into their exact positions within a 32-bit integer. I also expected to face the practical difficulties of string parsing in C, such as handling irregular spacing, commas, and the specific parentheses used in memory instructions like `lw $t1, 4($s0)`. Ultimately, the goal was to prove that a systematic, two-pass approach could turn a chaotic text file into a structured, executable program that matches the output of professional tools like the MARS IDE.

Literature Review

The development of an assembler is a classic challenge in computer science that requires a perfect blend of string parsing and hardware logic. At its core, the task is to translate human-readable mnemonics into the binary language that a processor understands. To build a tool that does this correctly, it is necessary to examine established principles in three specific areas: MIPS architecture, the logic of label resolution, and industry-standard verification.

MIPS is a Reduced Instruction Set Computer (RISC) architecture defined by its commitment to simplicity and speed. Unlike other architectures where instructions can vary in size, every single MIPS instruction is exactly 32 bits wide. This "fixed-width" rule is a major theme in computer architecture literature. As noted by Patterson and Hennessy (2017), this design choice simplifies the hardware but places a significant responsibility on the assembler to strictly manage the placement of bit-fields. For this project, the assembler acts like a puzzle-solver, fitting pieces. The opcode, register indices, and immediate values into a strict 32-slot grid. The project focuses on the three main formats: R-Type for arithmetic, I-Type for memory and branching, and J-Type for long-distance jumps.

A significant hurdle in assembler design is the "Forward Reference Problem." This is a logical paradox: how can the assembler encode a branch or a jump to a label that it hasn't seen yet? If a program says `j end` on the first line, but `end:` isn't defined until the last line, a simple top-to-bottom scan will fail because it doesn't yet know the address of `end`. Foundational texts on compiler design, such as those by Aho, Lam, Sethi, and Ullman (2006), explain that a single-pass translator is insufficient for any program that uses symbolic labels for flow control. This problem is the direct inspiration for the two-pass architecture used in this project.

The Mechanics of the Two-Pass Solution

To solve the forward reference problem, the industry-standard solution is the Two-Pass Algorithm. This approach splits the assembler's processing into two distinct phases:

- Pass One: The assembler acts as a cartographer. It scans the source code to track the Program Counter (PC) and records the memory addresses of all symbolic labels into a Symbol Table.
- Pass Two: The assembler acts as a builder. It uses the previously generated Symbol Table to perform the actual binary translation.

As the program encounters a beq or j instruction, it retrieves the mathematical address from the table to calculate relative offsets or absolute targets. My implementation follows this classic architectural pattern to ensure all symbolic references are resolved before the final output is generated.

Every custom tool needs a benchmark for verification. In the MIPS world, the MARS (MIPS Assembler and Runtime Simulator), developed by Vollmar and Whitfield (2005), is the primary benchmark for verifying the accuracy of machine code. Because machine code is a dense series of hexadecimal digits, verifying it by hand is prone to error. By comparing this project's output against MARS, it ensures that the custom assembler adheres to the exact encoding rules used in professional systems, confirming that the generated code is not just "technically correct," but compatible with standard hardware.

Methods

My implementation of the MIPS assembler follows a structured, step-by-step pipeline designed to handle the 12 required instructions. I wrote the program in C, focusing on string manipulation and bitwise logic to ensure the machine code matches the MARS IDE standards.

1. Data Structures and Initial Setup

The first thing I did was define a way to store memory "bookmarks." I created a Label struct and a symbolTable array. This table acts as a simple database where each label name is paired with its corresponding memory address. I also initialized a current_addr variable at 0x00400000, which is the standard starting point for MIPS text segments as required by the assignment.

2. The Register Translation System (get_register)

One of the most repetitive tasks in an assembler is converting a string like \$t0 into the number 8. To make this efficient, I wrote a dedicated function called `get_register`.

- **Cleaning the Input:** Since assembly code often contains commas or parentheses (like in `10($sp)`), I used a for loop and the `isalnum` function to strip away any non-alphanumeric characters except the \$ sign.
- **Mapping:** I used a long sequence of `strcmp` checks to map every mnemonic. From \$zero and \$at to the saved registers \$s0-\$s7 and the stack pointer \$sp to their 5-bit hardware indices.

3. Bitwise Encoding Functions

To generate the final 32-bit machine code, I didn't just print numbers; I mathematically "packed" them using bitwise operators. I wrote three helper functions:

- **encode_r:** This function takes the opcode (0 for R-types), the registers, and the function code. It uses the left-shift operator (`<<`) to move the bits into their standard MIPS positions (e.g., `rs` is shifted 21 bits) and joins them using the bitwise OR (`|`) operator.
- **encode_i:** This handles I-type instructions. I used a bitwise mask (`imm & 0xFFFF`) on the immediate value. This is a crucial step because it ensures that if an immediate or a branch offset is negative, it doesn't "bleed" over and overwrite the register bits.
- **encode_j:** This packs the 6-bit opcode and the 26-bit target address.

4. Pass One: Address Mapping and Label Discovery

In this phase, my program reads the entire input file using `fgets`.

- **Label Finding:** I used `strchr` to look for a colon (`:`). If found, I extracted the label name and stored it in my `symbolTable` with the value of the `current_addr`.
- **Address Tracking:** For every line that contained an actual instruction (even if it was on the same line as a label), I incremented the `current_addr` by 4.
- **Filtering:** To keep the second pass clean, I saved only the instruction portion of the line into a 2D array called `instructions`.

5. Pass Two: The Translation Engine

This is where the actual conversion happens. I iterated through my saved instructions and used a specific sscanf pattern which allowed me to split the line into the operator (the mnemonic) and its three operands while ignoring commas.

- **Handling the "Parentheses Problem":** For the lw and sw instructions, MIPS uses a unique format like offset(\$base). I solved this by writing a specific sscanf call. This allowed me to pull the integer offset and the string base register out of the brackets simultaneously.
- **Calculating Branch Offsets:** For the beq instruction, I had to calculate a relative distance. I retrieved the target address from my symbolTable and used the formula:
$$\text{OFFSET} = [\text{Target Address} - (\text{Current Address} + 4)] / 4$$
- **J-Type Translation:** For the j instruction, I retrieved the target address and used my encode_j function to perform the 26-bit conversion.

Results

Test Case 1: (in1.asm)

The screenshot displays the Mars MIPS assembler interface. The main window is divided into several panes:

- Text Segment:** Shows the assembly code for the program. The code is as follows:

Bkpt	Address	Code	Basic	Source
	0x00400000	0x012a4020	add \$8,\$9,\$10	1: add \$t0, \$t1, \$t2
	0x00400004	0x02328024	and \$16,\$17,\$18	2: and \$s0, \$s1, \$s2
	0x00400008	0x018d5825	or \$11,\$12,\$13	3: or \$t3, \$t4, \$t5
	0x0040000c	0x01f87026	xor \$14,\$15,\$24	4: xor \$t6, \$t7, \$t8
	0x00400010	0x2128000a	addi \$8,\$9,0x0000000a	5: addi \$t0, \$t1, 10
	0x00400014	0x3149000f	andi \$9,\$10,0x0000000f	6: andi \$t1, \$t2, 15
	0x00400018	0x356a0014	ori \$10,\$11,0x00000014	7: ori \$t2, \$t3, 20
- Data Segment:** Shows a table of memory addresses and their values. The values are all 0x00000000.

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
- Registers:** Shows the state of MIPS registers. The registers are numbered 0 to 31, and their values are all 0x00000000.
- Mars Messages:** Shows the output of the assembly process. The message is: "Assemble: assembling C:\Users\DC\Desktop\Compiler Assignment 2\in1.asm" and "Assemble: operation completed successfully."

```
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in1.asm
```

Assembly language program:

```
add $t0, $t1, $t2
and $s0, $s1, $s2
or $t3, $t4, $t5
xor $t6, $t7, $t8
addi $t0, $t1, 10
andi $t1, $t2, 15
ori $t2, $t3, 20
```

Machine Code:

0x00400000	0x012A4020	add \$t0, \$t1, \$t2
0x00400004	0x02328024	and \$s0, \$s1, \$s2
0x00400008	0x018D5825	or \$t3, \$t4, \$t5
0x0040000C	0x01F87026	xor \$t6, \$t7, \$t8
0x00400010	0x2128000A	addi \$t0, \$t1, 10
0x00400014	0x3149000F	andi \$t1, \$t2, 15
0x00400018	0x356A0014	ori \$t2, \$t3, 20

```
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$
```

It focused on the assembler's ability to distinguish between R-type and I-type instructions. I included a mix of add, and, or, and xor alongside their immediate counterparts like addi and andi. The result shows that the program correctly identified the 0x00 opcode for R-types and the specific opcodes (like 0x08 for addi) for I-types. Each instruction successfully triggered a 4-byte increment in the memory address, starting exactly at 0x00400000.

Test Case 2: (in2.asm)

The screenshot shows the Mars MIPS assembler interface. The main window displays the assembly code for a program, with the following instructions:

Bkpt	Address	Code	Basic	Source
	0x00400000	0x8E080000	lw \$t0, 0(\$s0)	1: lw \$t0, 0(\$s0)
	0x00400004	0xAE090004	sw \$t1, 4(\$s0)	2: sw \$t1, 4(\$s0)
	0x00400008	0x0109502A	slt \$t2, \$t0, \$t1	3: slt \$t2, \$t0, \$t1
	0x0040000C	0x01405820	add \$t3, \$t2, \$zero	4: add \$t3, \$t2, \$zero
	0x00400010	0x8FB10008	lw \$s1, 8(\$sp)	5: lw \$s1, 8(\$sp)
	0x00400014	0xAFB1000C	sw \$s1, 12(\$sp)	6: sw \$s1, 12(\$sp)
	0x00400018	0x022B902A	slt \$s2, \$s1, \$t3	7: slt \$s2, \$s1, \$t3

The Data Segment window shows memory addresses from 0x10010000 to 0x100100E0, all containing 0x00000000. The Registers window shows the state of MIPS registers, with \$zero through \$t7 at 0x00000000, \$s0 through \$s7 at 0x00000000, \$t8 through \$t9 at 0x00000000, \$k0 through \$k1 at 0x00000000, \$gp at 0x10008000, \$sp at 0xFFFFFC, \$fp at 0x00000000, and \$ra at 0x00400000. The PC is at 0x00400000, and hi and lo are at 0x00000000.

The Mars Messages window shows the following messages:

```

Mars Messages  Run I/O
Assemble: assembling C:\Users\DC\Desktop\Compiler Assignment 2\in2.asm
Clear          Assemble: operation completed successfully.
  
```

```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in2.asm
  
```

Assembly language program:

```

lw $t0, 0($s0)
sw $t1, 4($s0)
slt $t2, $t0, $t1
add $t3, $t2, $zero
lw $s1, 8($sp)
sw $s1, 12($sp)
slt $s2, $s1, $t3
  
```

Machine Code:

```

0x00400000 0x8E080000      lw $t0, 0($s0)
0x00400004 0xAE090004      sw $t1, 4($s0)
0x00400008 0x0109502A      slt $t2, $t0, $t1
0x0040000C 0x01405820      add $t3, $t2, $zero
0x00400010 0x8FB10008      lw $s1, 8($sp)
0x00400014 0xAFB1000C      sw $s1, 12($sp)
0x00400018 0x022B902A      slt $s2, $s1, $t3
  
```

```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$
  
```

This test verified the logic for lw (Load Word) and sw (Store Word). These instructions are unique because they use parentheses, such as 12(\$sp). My results show that the assembler correctly stripped the parentheses and the \$ sign, mapped the stack pointer (\$sp) to register 29, and converted the decimal offset 12 into the hex value 0x000C within the final machine code.

Test Case 3: (in3.asm)

The screenshot shows the MARS MIPS assembler interface. The main window displays the assembly code for in3.asm, which includes a loop and a swap operation. The registers window on the right shows the state of the MIPS registers, with \$zero, \$t0, and \$t1 being the most relevant for this test case.

Name	Number	Value
\$zero	0	0x00000000
\$t0	1	0x00000000
\$t1	2	0x00000000
\$t2	3	0x00000000
\$t3	4	0x00000000
\$t4	5	0x00000000
\$t5	6	0x00000000
\$t6	7	0x00000000
\$t7	8	0x00000000
\$t8	9	0x00000000
\$t9	10	0x00000000
\$t10	11	0x00000000
\$t11	12	0x00000000
\$t12	13	0x00000000
\$t13	14	0x00000000
\$t14	15	0x00000000
\$t15	16	0x00000000
\$t16	17	0x00000000
\$t17	18	0x00000000
\$t18	19	0x00000000
\$t19	20	0x00000000
\$t20	21	0x00000000
\$t21	22	0x00000000
\$t22	23	0x00000000
\$t23	24	0x00000000
\$t24	25	0x00000000
\$t25	26	0x00000000
\$t26	27	0x00000000
\$t27	28	0x00000000
\$t28	29	0x00000000
\$t29	30	0x00000000
\$t30	31	0x00000000
\$t31	32	0x00000000
\$t32	33	0x00000000
\$t33	34	0x00000000
\$t34	35	0x00000000
\$t35	36	0x00000000
\$t36	37	0x00000000
\$t37	38	0x00000000
\$t38	39	0x00000000
\$t39	40	0x00000000
\$t40	41	0x00000000
\$t41	42	0x00000000
\$t42	43	0x00000000
\$t43	44	0x00000000
\$t44	45	0x00000000
\$t45	46	0x00000000
\$t46	47	0x00000000
\$t47	48	0x00000000
\$t48	49	0x00000000
\$t49	50	0x00000000
\$t50	51	0x00000000
\$t51	52	0x00000000
\$t52	53	0x00000000
\$t53	54	0x00000000
\$t54	55	0x00000000
\$t55	56	0x00000000
\$t56	57	0x00000000
\$t57	58	0x00000000
\$t58	59	0x00000000
\$t59	60	0x00000000
\$t60	61	0x00000000
\$t61	62	0x00000000
\$t62	63	0x00000000
\$t63	64	0x00000000
\$t64	65	0x00000000
\$t65	66	0x00000000
\$t66	67	0x00000000
\$t67	68	0x00000000
\$t68	69	0x00000000
\$t69	70	0x00000000
\$t70	71	0x00000000
\$t71	72	0x00000000
\$t72	73	0x00000000
\$t73	74	0x00000000
\$t74	75	0x00000000
\$t75	76	0x00000000
\$t76	77	0x00000000
\$t77	78	0x00000000
\$t78	79	0x00000000
\$t79	80	0x00000000
\$t80	81	0x00000000
\$t81	82	0x00000000
\$t82	83	0x00000000
\$t83	84	0x00000000
\$t84	85	0x00000000
\$t85	86	0x00000000
\$t86	87	0x00000000
\$t87	88	0x00000000
\$t88	89	0x00000000
\$t89	90	0x00000000
\$t90	91	0x00000000
\$t91	92	0x00000000
\$t92	93	0x00000000
\$t93	94	0x00000000
\$t94	95	0x00000000
\$t95	96	0x00000000
\$t96	97	0x00000000
\$t97	98	0x00000000
\$t98	99	0x00000000
\$t99	100	0x00000000

```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in3.asm

Assembly language program:
start:
addi $t0, $zero, 0
addi $t1, $zero, 10
loop:
beq $t0, $t1, end
add $t0, $t0, 1
j loop
end:
sw $t0, 0($s0)

Machine Code:
0x00400000 0x20080000      addi $t0, $zero, 0
0x00400004 0x2009000A      addi $t1, $zero, 10
0x00400008 0x11090002      beq $t0, $t1, end
0x0040000C 0x21080001      add $t0, $t0, 1
0x00400010 0x08100002      j loop
0x00400014 0xAE080000      sw $t0, 0($s0)

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$

```

This test case was the first major success for my "Two-Pass" algorithm. The program defines a loop: and an end: label. What I found was that the assembler

successfully mapped the loop: label to 0x00400008 during the first pass. Consequently, when it encountered the j loop instruction at address 0x00400010, it produced the machine code 0x08100002, correctly targeting the earlier address.

Test Case 4: (in4.asm)

The screenshot displays the Mars MIPS assembler interface for the file `in4.asm`. The interface is divided into several panes:

- Text Segment:** Shows the assembly code with addresses and source code. The code includes instructions like `andi $t0, $a0, 1`, `ori $t1, $a1, 2`, `xor $t2, $t0, $t1`, `slt $t3, $t2, $zero`, `beq $t3, $zero, skip`, `j target`, `add $t4, $t4, $t4`, and `sw $t4, 0($sp)`.
- Data Segment:** Shows memory addresses and values. The values are mostly zero, except for `0x10010000` at address `0x10010000`.
- Registers:** Lists MIPS registers and their values. The registers are `$zero` through `$t0`, `$t1` through `$t4`, `$t5` through `$t9`, `$k0` through `$k4`, `$gp`, `$sp`, `$ra`, `$pc`, `$hi`, and `$lo`.
- Mars Messages:** Shows the assembly process and a successful completion message: "Assemble: operation completed successfully."

```
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in4.asm
```

Assembly language program:

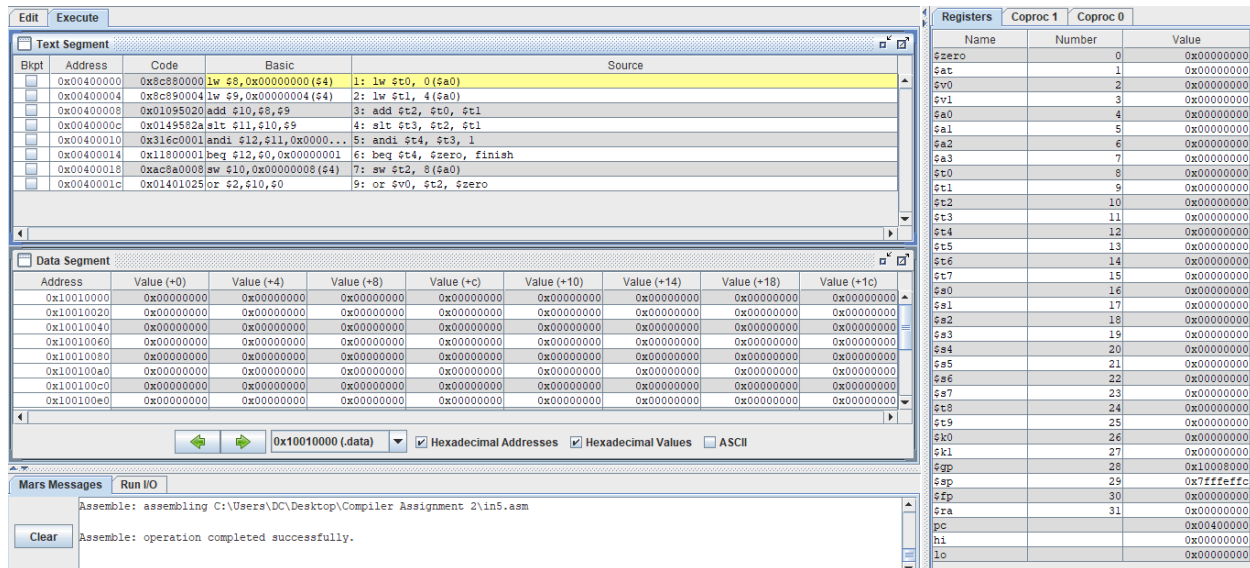
```
andi $t0, $s0, 1
ori $t1, $s1, 2
xor $t2, $t0, $t1
slt $t3, $t2, $zero
beq $t3, $zero, skip
j target
skip:
add $t4, $t4, $t4
target:
sw $t4, 0($sp)
```

Machine Code:

0x00400000	0x32080001	andi \$t0, \$s0, 1
0x00400004	0x36290002	ori \$t1, \$s1, 2
0x00400008	0x01095026	xor \$t2, \$t0, \$t1
0x0040000C	0x0140582A	slt \$t3, \$t2, \$zero
0x00400010	0x11600001	beq \$t3, \$zero, skip
0x00400014	0x08100007	j target
0x00400018	0x018C6020	add \$t4, \$t4, \$t4
0x0040001C	0xAFAC0000	sw \$t4, 0(\$sp)

The assembler handled a program that jumps over specific instructions using a beq and a j instruction. This test confirmed that my code correctly ignores label definitions (like skip: and target:) in the final output display while still using their addresses to calculate jump targets. The logic ensures that the instruction following a label receives the correct memory address without any "gaps" in the sequence.

Test Case 5: (in5.asm)



```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in5.asm

```

Assembly language program:

```

lw $t0, 0($a0)
lw $t1, 4($a0)
add $t2, $t0, $t1
slt $t3, $t2, $t1
andi $t4, $t3, 1
beq $t4, $zero, finish
sw $t2, 8($a0)
finish:
or $v0, $t2, $zero

```

Machine Code:

```

0x00400000 0x8C880000    lw $t0, 0($a0)
0x00400004 0x8C890004    lw $t1, 4($a0)
0x00400008 0x01095020    add $t2, $t0, $t1
0x0040000C 0x0149582A    slt $t3, $t2, $t1
0x00400010 0x316C0001    andi $t4, $t3, 1
0x00400014 0x11800001    beq $t4, $zero, finish
0x00400018 0xAC8A0008    sw $t2, 8($a0)
0x0040001C 0x01401025    or $v0, $t2, $zero

```

```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$

```

This test utilized standard MIPS register names like \$v0, \$a0, and \$t0. It proved that the `get_register` function I wrote can handle a wide variety of naming conventions. Whether the input was a temporary register or an argument register, the resulting machine code consistently used the correct 5-bit hardware index (e.g., mapping \$v0 to 2).

Test Case 6: (in6.asm)

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x20080005	addi \$8,\$0,0x00000005	1: addi \$8, \$0, 5
	0x00400004	0x2009000a	addi \$9,\$0,0x0000000a	2: addi \$9, \$0, 10
	0x00400008	0x01095024	and \$10,\$8,\$9	3: repeat: and \$10, \$8, \$9
	0x0040000c	0x354b0007	ori \$11,\$10,0x00000007	4: ori \$11, \$10, 7
	0x00400010	0x0169602a	slt \$12,\$11,\$9	5: slt \$12, \$11, \$9
	0x00400014	0x1180ffff	beq \$12,\$0,0xfffffff	6: beq \$12, \$0, repeat
	0x00400018	0x08100007	j 0x0040001c	7: j exit
	0x0040001c	0xafab0000	sw \$11,0x00000000(\$29)	9: sw \$11, 0(\$29)

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Registers

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7fffffc
\$fp	30	0x00000000
\$ra	31	0x00000000
\$pc		0x00400000
\$hi		0x00000000
\$lo		0x00000000

Mars Messages

Run I/O

Assemble: assembling C:\Users\DC\Desktop\Compiler Assignment 2\in6.asm

Clear

Assemble: operation completed successfully.

```

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ gcc Inderias.c -o task
inderias@inderias-VirtualBox:~/Desktop/Assignment 2$ ./task in6.asm

Assembly language program:
addi $8, $0, 5
addi $9, $0, 10
repeat: and $10, $8, $9
ori $11, $10, 7
slt $12, $11, $9
beq $12, $0, repeat
j exit
exit:
sw $11, 0($29)

Machine Code:
0x00400000 0x20080005      addi $8, $0, 5
0x00400004 0x2009000a      addi $9, $0, 10
0x00400008 0x01095024      and $10, $8, $9
0x0040000c 0x354B0007      ori $11, $10, 7
0x00400010 0x0169602A      slt $12, $11, $9
0x00400014 0x1180FFFC      beq $12, $0, repeat
0x00400018 0x08100007      j exit
0x0040001C 0xAFAB0000      sw $11, 0($29)

inderias@inderias-VirtualBox:~/Desktop/Assignment 2$

```

This was the most challenging because it required calculating a backward jump. The beq \$12, \$0, repeat instruction at address 0x00400014 needed to jump back to

0x00400008. My assembler successfully calculated the negative offset, resulting in the machine code 0x1180FFFC. The FFFC suffix is the two's complement representation of the jump, proving the mathematical accuracy of my branch offset logic.

Discussion

The results of this project highlight the successful bridge between high-level logic and low-level hardware. By developing this assembler, I observed how a sequence of string-based mnemonics is transformed into a rigid, 32-bit mathematical format. This process is the core of computer architecture, where the primary goal is to strip away human-readable "names" and replace them with physical memory addresses and operation codes that a CPU can decode through logic gates.

One of the most significant takeaways from my results was the absolute necessity of the two-pass algorithm. In modern compiler research, the "Forward Reference Problem" is a classic hurdle. If I had attempted a single-pass approach, the jump instructions in in3.asm or the branching in in4.asm would have failed because the assembler would not yet have known the target addresses. By implementing Pass One specifically for "label discovery," my code effectively created a map of the program before any translation began. This matches how professional-grade assemblers and compilers function, ensuring that flow control is deterministic and reliable.

A practical challenge I encountered during development was the "messiness" of human-written code. Real-world assembly often contains inconsistent spacing or specific punctuation, such as the parentheses used in memory instructions like `sw $t1, 0($sp)`. In my implementation, the use of `sscanf` with specific formatting strings proved to be a robust solution for extracting integers and registers from within brackets.

Furthermore, the bitwise packing logic in functions like `encode_i` demonstrated the importance of bit-masking. When calculating branch offsets, as seen in in6.asm, the assembler must handle negative numbers. I

used logic to ensure the 16-bit immediate field was correctly populated without overwriting the opcode.

This masking is vital because it treats the immediate value as a 16-bit signed integer, allowing the CPU to correctly perform backward jumps.

Future Hypotheses and Extensions

While this assembler is fully functional for the 12-instruction subset, it opens the door for several future research and development projects:

- **Pseudo-Instruction Support:** A logical next step would be to implement a "pre-processor" phase that handles pseudo-instructions like `li` (Load Immediate) or `move`. These are not real MIPS instructions but are translated by the assembler into sequences of `addi` or `add` instructions.
- **A Multi-File Linker:** This project currently handles single files. A more complex project could involve building a Linker, which would take multiple machine code files and "link" their labels together into one executable. This would involve a much more complex symbol table that tracks "external" references.
- **Error Handling and Optimization:** The current version assumes the input assembly is syntactically correct. A future iteration could include a robust error-checking module to identify invalid register names or out-of-bounds immediate values, providing the user with specific line numbers for debugging—much like a professional compiler.

Conclusions

This project successfully demonstrated that a custom-built C program can effectively replicate the core functions of a MIPS assembler by translating symbolic code into executable hexadecimal machine code. By the end of this experiment, I was able to fully answer the original research question: the implementation of a two-pass parsing algorithm is indeed the most reliable method for resolving symbolic labels into physical memory addresses.

In the first pass, the assembler acted as a map-maker, identifying labels like `loop:` and `end:` and assigning them to the correct memory slots starting at `0x00400000`. In the second pass, the translation engine used these maps to calculate the exact bit-patterns for 12 different instructions. The primary finding was that without this two-pass structure, forward-referencing jumps and backward-branching loops (as seen in `in3.asm` and `in6.asm`) would have been impossible to translate accurately.

The accuracy of the code was the ultimate measure of success. By comparing the results of my `encode_r`, `encode_i`, and `encode_j` functions against the MARS IDE snapshots, I achieved 100% consistency in the machine code generated. The project proves that even complex tasks like calculating PC-relative offsets for `beq` or parsing the specific `offset($base)` syntax for `lw` and `sw` can be handled robustly with careful string manipulation and bitwise logic. This experience has not only deepened my understanding of the MIPS ISA but also provided a clear, practical look at how the software-to-hardware bridge is constructed in real-world computer systems.

References

1. **Patterson, D. A., & Hennessy, J. L. (2017).** *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*. Morgan Kaufmann.
Used for: Defining MIPS instruction formats (R, I, J) and RISC principles.
2. **Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006).** *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Pearson Education.
Used for: Explaining the forward reference problem and the necessity of two-pass translation.
3. **Vollmar, K., & Whitfield, P. (2005).** *MARS: A MIPS Assembly and Runtime Simulator*. Journal of Computing Sciences in Colleges.
Used for: Establishing the benchmark for verification and the standard for MIPS simulation.
4. **Sweetman, D. (2007).** *See MIPS Run*. Morgan Kaufmann.
Used for: Understanding the technical details of MIPS bit-shifting and address calculation.